

AI 协作反思日志 — Phase 4 核心开发与功能完善

团队：第44组

日期：2026年6月1日

1. 本阶段 AI 使用清单

本阶段团队主要使用 Cursor 和 Codex 辅助开发、测试和调试工作。AI 主要参与用户权限模块、任务管理模块、标注工作台、裁定与导出功能的实现，以及测试用例编写和 CI 配置等工作。

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
用户权限模块	Cursor	实现 JWT 登录、拦截器与前端路由守卫	4	30
任务管理模块	Cursor	生成任务创建、成员分配和状态推进逻辑	3	55
标注工作台	Cursor, Codex	划词生成命题、标签选择与自动编号	4	35
关系标注功能	Cursor	关系编辑界面与数据持久化	3	50
论证图生成	Cursor, Codex	Vue Flow 与 ELK 自动布局集成	3	60
裁定页面	Cursor	双标注结果对比与裁定逻辑	3	50
导出功能	Cursor	JSON、ZIP、图示导出功能	3	40
Mapper与XML生成	Cursor	MyBatis Mapper与XML模板	3	55
Sprint规划	Cursor	任务拆解与工时估算	4	25
Bug定位与修复	Cursor, Codex	权限、跨域和认证问题排查	4	20
测试用例生成	Cursor	主流程测试场景设计	3	40
GitLab CI配置	Cursor	自动构建与测试脚本生成	4	15

2. AI 助力分析（Q1）

AI 在编码中最有价值的场景

本阶段 AI 对于样板代码生成帮助最明显。例如 Controller、Service、Mapper 三层结构、Vue 表单组件以及 Axios 请求封装等内容，AI 能快速生成基本框架，使团队能够将更多时间投入到业务逻辑实现中。

此外，在 Vue Flow、ELK、Element Plus 等第三方库集成过程中，AI 能直接给出示例代码，减少查阅文档的时间成本。

对于涉及多个文件的功能开发，AI 能够快速列出需要修改的前后端文件，帮助团队梳理改动范围。

AI 在测试生成中最有价值的场景

AI 能快速构建主流程测试场景，例如：

管理员创建用户 → 创建任务 → 多人标注 → 裁定 → 导出结果。

同时，AI 能枚举大量边界测试场景，如阶段不可逆、重复标注、权限越权等问题，为团队完善测试用例提供参考。

AI 在调试中最有价值的场景

AI 对于编译错误、配置问题和常见框架问题的定位效率较高。

例如：

- Spring Boot 配置错误
- Vue 组件导入缺失
- MyBatis 映射异常
- JWT Token 校验问题
- CORS 跨域配置问题

这些问题通常能够通过 AI 提供的方案快速解决。

3. AI 误导分析（Q2）

AI 代码信任度实验

为了评估 AI 输出代码的可靠性，团队随机抽取了 20 份 AI 生成的代码或测试样例进行验证。

实验结果如下：

- AI 直出的测试通过率：35%
- 人工审查修复后的测试通过率：90%

说明 AI 生成内容可以作为开发起点，但不能直接作为最终结果使用。

发现的主要问题类型包括：

1. 业务规则理解不准确
2. 权限校验遗漏
3. 与项目现有代码风格不一致
4. 边界条件处理不足

5. 过度设计或引入不必要依赖

典型问题举例：

- AI 生成的代码未校验「所有标注员已提交标注结果」，阶段可被非法推进。
- AI 补全的标注提交接口未校验「当前用户是否为任务标注员」，存在越权提交风险。
- AI 生成的论证图布局方案在复杂关系场景下效果较差，出现节点重叠、边交叉过多等问题，需要人工调整 ELK 参数和布局规则。

AI调试对决实验

团队选取多个真实 Bug 分别由 AI 和人工进行分析。

结果发现：

AI 擅长定位：

- 编译失败
- 配置错误
- 依赖缺失
- MyBatis 映射问题
- Vue 组件问题

AI 不擅长定位：

- 业务状态机错误
- 权限边界问题
- 多角色协作流程问题
- 数据一致性问题

AI 给出的修复方案常见问题包括：

- 通过新增字段或新增表解决问题
- 在前端绕过权限而非后端校验
- 引入不必要的抽象层
- 修复当前问题但影响其他业务流程

因此团队形成共识：

配置类问题优先借助 AI 排查；涉及业务逻辑、权限控制和状态流转的问题必须由开发人员主导分析。

4. Prompt 改进计划（Q3）

上阶段改进措施验证结果

本阶段验证表明：

- 强调职责分离后，AI 生成的代码结构更加合理；
- 强制要求权限校验后，大部分接口具备鉴权逻辑；
- 要求输出改动文件列表后，遗漏问题明显减少；

- 明确“课程项目”背后，AI 不再频繁推荐微服务等过度设计方案。

总体来看，上阶段 Prompt 优化取得了较好的效果。

编码阶段 Prompt 最佳实践

团队总结出以下经验：

1. 明确技术栈与项目结构；
2. 给出相关代码上下文；
3. 强调最小修改原则；
4. 按步骤逐步生成代码；
5. 明确验收标准与边界条件。

调试阶段 Prompt 最佳实践

1. 提供完整报错信息；
2. 说明复现步骤；
3. 告知已尝试方案；
4. 限定允许修改范围；
5. 要求分析根因而非仅给出修改代码。

下阶段改进计划

下一阶段团队计划：

1. 建立统一 Prompt 模板库；
2. 增加数据库变更约束说明；
3. 测试 Prompt 强制覆盖正常、边界和越权场景；
4. 记录 AI 原始输出与人工修正结果，用于持续评估 AI 输出质量。

5. 本阶段的核心工程判断

决策内容

(1) 论证图采用「手动点击生成」，而非「标注变更实时自动刷新」

- P0 优先保证命题与关系数据正确入库。
- 图示作为展示层，允许与标注数据短暂不同步。
- 降低 Vue Flow + ELK 在复杂关系下的联动 Bug 风险。

(2) 任务身份采用「全局角色 + 任务级成员身份」双轨制

- 全局：admin / creator / user + canCreateTask 控制菜单与后台能力。
- 任务级：task_member.role_in_task 决定标注员 / 裁决者行为。
- 避免把「标注者 / 裁决者」做成全局角色，符合 SRS 与 P1 调研结论。

为什么 AI 无法单独做这个决策

- 1. 业务优先级需要产品判断：AI 倾向于实现 SRS 字面要求的实时双向同步图示，但团队清楚现有工期下应优先能演示完整闭环；可交付性 > 交互完美性 的排序 AI 无法自主得出。
- 2. 角色模型需要领域理解：AI 常把标注员/裁决者设计成全局 Role 枚举，不理解本系统同一 user 在不同任务中身份不同的双轨制，易导致权限模型整体返工。
- 3. 集成风险需要联调经验：AI 不知道 Vue Flow + ELK 集成代码复杂、多成员关系下节点重叠严重；手动生成是基于已发生成本的判断。

团队最终如何判断和取舍

决策点	备选方案	最终选择	取舍依据
图示刷新	实时自动 / 手动生成	手动生成	联调风险低；与 README 行为一致
角色模型	全局四角色 / 双轨制	双轨制	符合 SRS；避免 creator 又是标注员
AI 代码合入	直出即用 / 必审查	必审查 + 主链路自测	信任度实验：直出通过率仅 35%